

Machine Learning II: Advanced Data Balancing Techniques

Week 6, Day 1: Transitioning to SMOTE and Imbalanced Learning

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

15th September 2026

Abstract

This presentation advances our classification journey by introducing robust data balancing techniques to handle class imbalance in clinical datasets.

- **The Imbalance Problem:** Understanding why standard accuracy fails and the clinical cost of False Negatives.
- **Balancing Techniques:** Overview of undersampling, oversampling, and the Synthetic Minority Over-sampling Technique (SMOTE).
- **Leak-Proof Implementation:** Integrating SMOTE into `imblearn` Pipelines to prevent data leakage during cross-validation.
- **Evaluation:** Comparing SMOTE-augmented ensemble models against baselines using Recall, F1, ROC-AUC, and PR-AUC.

Session Objectives

- Diagnose class imbalance and its impact on model evaluation metrics.
- Understand the mathematical intuition behind SMOTE.
- Implement SMOTE safely using `imblearn.pipeline.Pipeline` to avoid data leakage.
- Update hyperparameter grids by removing redundant `class_weight` parameters.
- Train and tune six ensemble classifiers on SMOTE-balanced training folds.
- Evaluate and justify the final model based on the clinical trade-off between Sensitivity (Recall) and Precision.

Part 1: The Problem of Imbalance

"Why standard metrics fail in the real world."

What is Class Imbalance?

- **Definition:** A situation in classification where the target variable has a highly skewed distribution of classes.
- **Example:** In fraud detection, 99.9% of transactions are legitimate, and 0.1% are fraudulent.
- **Our Dataset:** The Heart Disease dataset is moderately imbalanced (54% No Disease, 46% Disease), but in real-world county health facilities, severe disease cases are often the minority.
- **The Danger:** Models trained on imbalanced data tend to become biased toward the majority class, ignoring the minority class entirely.

The Accuracy Paradox

The 99% Accuracy Illusion

Imagine a model predicting a rare disease (1% prevalence).
A "dumb" model that always predicts “No Disease” will achieve **99% Accuracy**.

- **Accuracy** is misleading when classes are imbalanced.
- We must shift focus to metrics that evaluate the minority class:
  - **Recall (Sensitivity):** Out of all actual sick patients, how many did we catch?
  - **Precision:** Out of all patients we flagged as sick, how many are actually sick?
  - **PR-AUC:** Precision-Recall Area Under Curve (superior to ROC-AUC for severe imbalance).

The Clinical Cost of Imbalance

False Negatives (Type II Error)

Missed cardiac risk.

Impact: Patient sent home, potential acute myocardial infarction (MI) or stroke.

Cost: HIGH (Loss of life). Recall must be prioritized.

False Positives (Type I Error)

Unnecessary angiography/stress tests.

Impact: Patient anxiety, wasted healthcare resources, financial burden.

Cost: MODERATE. Precision balances this.

Part 2: Data Balancing Techniques

"Leveling the playing field for the minority class."

Overview of Balancing Techniques

- **Algorithmic Level:** Modify the learning algorithm (e.g., `class_weight='balanced'`, custom loss functions). *We used this in Week 5.*
- **Data Level:** Modify the training dataset to balance class distributions before feeding it to the model.
  - **Undersampling:** Reduce the majority class.
  - **Oversampling:** Duplicate the minority class.
  - **Synthetic Oversampling:** Generate new, synthetic minority samples (e.g., SMOTE, ADASYN).

Best Practice

Data-level balancing should **only** be applied to the **training set**, never the validation or test sets.

Undersampling vs. Oversampling

Random Undersampling

Method: Randomly discard majority class samples.

Pros: Reduces training time; prevents model from being overwhelmed.

Cons: Loss of valuable information. May discard borderline or critical majority examples.

Random Oversampling

Method: Randomly duplicate minority class samples.

Pros: No loss of information; simple to implement.

Cons: High risk of overfitting. The model memorizes the exact duplicated instances.

SMOTE: Synthetic Minority Over-sampling Technique

The SMOTE Solution

Proposed by Chawla et al. (2002), SMOTE generates **synthetic** examples of the minority class rather than simply duplicating them.

- **How it works:**
  1. Select a minority class instance x_i.
  2. Find its k nearest neighbors (using Euclidean distance) within the minority class.
  3. Randomly select one neighbor x_{zi}.
  4. Create a synthetic sample x_{new} along the line segment between x_i and x_{zi}:
     $x_{new} = x_i + \lambda \times (x_{zi} - x_i)$, where $\lambda \in [0, 1]$.

The Danger of Data Leakage

The Critical Mistake

Applying SMOTE to the **entire dataset** *before* the train/test split.

- If SMOTE is applied globally, synthetic samples are created using information from the entire dataset.
- These synthetic samples will inevitably leak into the training, validation, and test sets.
- **Result:** The model is tested on data it has already "seen" in a synthetic form, leading to wildly inflated, unrealistic performance metrics (Data Leakage).

The Rule

SMOTE must ONLY be applied to the training folds during cross-validation.

The Solution: `imblearn` Pipelines

- Standard `sklearn.pipeline.Pipeline` does not support resampling steps natively.
- The `imbalanced-learn` (`imblearn`) library provides `imblearn.pipeline.Pipeline`.
- **How it prevents leakage:** During `RandomizedSearchCV`, the pipeline `fit` method is called on each training fold. SMOTE is dynamically applied *only* to that specific fold's training data, keeping the validation fold completely untouched and pristine.

Installation

```
!pip install imbalanced-learn
```

Part 3: Implementing SMOTE

"Coding a leak-proof balancing workflow."

Installing and Importing Libraries

Python Implementation

```
# Install the library (run once in notebook)
# !pip install imbalanced-learn

# Import SMOTE and the imblearn Pipeline
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

```

Code Explanation

- `SMOTE`: The core algorithm that generates synthetic minority samples.
- `ImbPipeline`: A specialized pipeline from `imblearn` that knows how to handle resampling steps safely during cross-validation, unlike the standard `sklearn` pipeline.

Inspecting Original Class Distribution

Python Implementation

```
# Initialize SMOTE with a fixed random state for reproducibility
smote = SMOTE(random_state=SEED, k_neighbors=5)

# 1. Check original class distribution
print("Original Training Set Class Distribution:")
print(y_train.value_counts())
print("-" * 50)

```

Code Explanation

- `random_state=SEED`: Ensures the random neighbor selection is reproducible.
- `k_neighbors=5`: SMOTE will look at the 5 nearest minority neighbors to generate synthetic points (default is 5).
- We print `y_train.value_counts()` to establish our baseline before balancing.

Applying SMOTE to Training Data

Python Implementation

```
# 2. Manually apply SMOTE to the preprocessed training data
# just to visualize the result
X_train_prep = preprocessor.fit_transform(X_train)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_prep, y_train)

print("Balanced Training Set Class Distribution (after SMOTE):")
print(pd.Series(y_train_resampled).value_counts())

```

Code Explanation

- `preprocessor.fit_transform(X_train)`: We first apply our numeric/categorical preprocessing to ensure SMOTE operates on scaled, encoded numerical data.
- `smote.fit_resample()`: This is the core SMOTE function. It takes the features and labels, and returns the augmented dataset where the minority class has been synthetically expanded to match the majority class count.

Part 4: SMOTE-Integrated Pipelines

"Updating our models for balanced learning."

Why Remove `class_weight='balanced'`?

The "Double-Balancing" Trap

In Week 5, we used `class_weight='balanced'` in Random Forest and XGBoost to handle imbalance.

- **The Problem:** If we use SMOTE (which balances the data by creating synthetic samples) *and* we also tell the model to apply class weights, the model will over-compensate.
- **The Result:** The model becomes overly biased toward the synthetic minority class, leading to a massive spike in False Positives and a drop in Precision.
- **The Solution:** When using SMOTE, we **remove** `class_weight` and `scale_pos_weight` parameters from the model definitions. SMOTE handles the balancing; the model just learns.

Building SMOTE Pipelines (Part 1: Trees)

Python Implementation

```
# Decision Tree Pipeline
dt_pipeline_smote = ImbPipeline(steps=[('preprocessor', preprocessor),
    ('smote', SMOTE(random_state=SEED)),
    ('dt', DecisionTreeClassifier(random_state=SEED)) # No class_weight!])

# Random Forest Pipeline
rf_pipeline_smote = ImbPipeline(steps=[('preprocessor', preprocessor),
    ('smote', SMOTE(random_state=SEED)),
    ('rf', RandomForestClassifier(random_state=SEED, n_jobs=-1)) # No class_weight!])

```

Code Explanation

We use `ImbPipeline`. The order is critical: `preprocessor` → `smote` → `classifier`. Notice that `class_weight='balanced'` has been explicitly removed from the classifiers.

Building SMOTE Pipelines (Part 2: Boosting)

Python Implementation

```
# AdaBoost Pipeline
ada_pipeline_smote = ImbPipeline(steps=[('preprocessor', preprocessor),
    ('smote', SMOTE(random_state=SEED)),
    ('ada', AdaBoostClassifier(random_state=SEED))])
# XGBoost Pipeline
xgb_pipeline_smote = ImbPipeline(steps=[('preprocessor',
</think>
preprocessor),'smote', SMOTE(random_state=SEED)),
    ('xgb', xgb.XGBClassifier(random_state=SEED, eval_metric='logloss', n_jobs=-1,
    scale_pos_weight=1)) # scale_pos_weight neutralized])

```

Code Explanation

For XGBoost, we explicitly set `scale_pos_weight=1` (the default for balanced data) to override any previous dynamic weight calculations, ensuring SMOTE is the sole balancing mechanism.

Part 5: Hyperparameter Tuning with SMOTE

"Optimizing models on balanced folds."

Updated Hyperparameter Grids (Part 1)

Python Implementation

```
# Updated Grid for Decision Tree (Removed class_weight)
param_dist_dt_smote = {
    'dt__max_depth': [None, 3, 5, 7, 10, 15, 20],
    'dt__min_samples_split': [2, 5, 10, 15],
    'dt__min_samples_leaf': [1, 2, 4, 6],
    'dt__criterion': ['gini', 'entropy', 'log_loss'],
    'dt__max_features': [None, 'sqrt', 'log2']
}

# Updated Grid for Random Forest (Removed class_weight)
param_dist_rf_smote = {
    'rf__n_estimators': [100, 200, 300, 400, 500],
    'rf__max_depth': [None, 5, 10, 15, 20, 25],
    'rf__min_samples_split': [2, 5, 10, 15],
    'rf__min_samples_leaf': [1, 2, 4, 6],
    'rf__max_features': ['sqrt', 'log2', None],
    'rf__criterion': ['gini', 'entropy', 'log_loss']
}

```

Code Explanation: Updated Grids (Part 1)

- **What changed:** The keys `'dt__class_weight'` and `'rf__class_weight'` have been completely removed from the dictionaries.
- **Why:** As established, SMOTE is now handling the class distribution balancing at the data level. Leaving `class_weight` in the grid would allow `RandomizedSearchCV` to accidentally select a balanced weight, causing the "double-balancing" effect and degrading Precision.
- **Impact:** The search space is slightly smaller, making the grid search marginally faster while focusing purely on structural tree parameters (depth, splits, features).

Key Takeaway

Hyperparameter grids must always align with the preprocessing strategy.

Updated Hyperparameter Grids (Part 2)

Python Implementation

```
# XGBoost Grid (scale_pos_weight removed/neutralized)
param_dist_xgb_smote = {
    'xgb__n_estimators': [100, 200, 300, 400, 500],
    'xgb__max_depth': [2, 3, 4, 5, 6],
    'xgb__learning_rate': [0.01, 0.03, 0.05, 0.1, 0.2],
    'xgb__subsample': [0.7, 0.8, 0.9, 1.0],
    'xgb__colsample_bytree': [0.7, 0.8, 0.9, 1.0],
    'xgb__min_child_weight': [1, 3, 5, 7],
    'xgb__gamma': [0, 0.1, 0.3, 0.5, 1],
    'xgb__reg_alpha': [0, 0.01, 0.1, 1],
    'xgb__reg_lambda': [0.1, 1, 5, 10]
}

```

Code Explanation: Updated Grids (Part 2)

- **XGBoost Specifics:** In the previous week, the grid included `'xgb__scale_pos_weight': [1, pos_weight_train]`.
- **The Update:** This key is removed from the grid. Furthermore, in the pipeline definition (Slide 21), we hardcoded `scale_pos_weight=1`.
- **Reasoning:** XGBoost's `scale_pos_weight` is mathematically similar to `class_weight` in sklearn. It adjusts the gradient calculation to penalize minority class errors more heavily. Since SMOTE already provides equal numbers of positive and negative examples to the model, the gradient should treat them equally (weight $= 1$).

Combining Pipelines for Search

Python Implementation

```
# Combine into a new list of pipelines
ensemble_pipelines_smote = [
    ('Decision Tree', dt_pipeline_smote, param_dist_dt_smote),
    ('Random Forest', rf_pipeline_smote, param_dist_rf_smote),
    ('Extra Trees', extra_trees_pipeline_smote, param_dist_extra_trees_smote),
    ('AdaBoost', ada_pipeline_smote, param_dist_ada_smote),
    ('Gradient Boosting', gb_pipeline_smote, param_dist_gb_smote),
    ('Extreme Gradient Boosting', xgb_pipeline_smote, param_dist_xgb_smote)
]

```

Code Explanation

We create a new list, `ensemble_pipelines_smote`, mirroring the structure from Week 5. This allows us to use the exact same `for` loop for `RandomizedSearchCV`, ensuring a fair, apples-to-apples comparison between the baseline (Week 5) and SMOTE-augmented (Week 6) models.

Executing RandomizedSearchCV

Python Implementation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
print("Starting RandomizedSearchCV with SMOTE (n_iter=30, 5-Fold)...")
results_ensemble_smote = {}
training_times_smote = {}
for name, pipeline, params in ensemble_pipelines_smote:
    start = time.time()

    search = RandomizedSearchCV(
        estimator=pipeline, param_distributions=params,
        n_iter=30, cv=cv, scoring='roc_auc',
        n_jobs=-1, random_state=SEED, refit=True
    )
    search.fit(X_train, y_train) # SMOTE is applied safely inside here!
    training_times_smote[name] = time.time() - start
    results_ensemble_smote[name] = search

```

Code Explanation: CV Execution

- **The Magic of `ImbPipeline`:** When `search.fit(X_train, y_train)` is called, `StratifiedKFold` splits the data into 5 folds.
- **Per-Fold Execution:** For each fold, the pipeline executes:
  1. `preprocessor.fit_transform()` on the 4 training folds.
  2. `smote.fit_resample()` **only on the preprocessed 4 training folds**.
  3. `classifier.fit()` on the SMOTE-augmented training folds.
  4. `classifier.predict()` on the untouched validation fold.
- **Result:** Zero data leakage. The validation fold remains completely original and unseen by the SMOTE algorithm.

Part 6: Evaluation & Results

"Does SMOTE actually improve clinical outcomes?"

Hold-Out Test Set Evaluation Setup

Python Implementation

```
eval_metrics_smote = {}
for name, search in results_ensemble_smote.items():
    model = search.best_estimator_
    # Predict on the PURE, unbalanced, unseen test set
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    # Calculate metrics
    recall = recall_score(y_test, y_pred, zero_division=0)
    precision = precision_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_prob)

    # False Negative Rate (Crucial for clinical context)
    fnr = fn / (fn + tp) if (fn + tp) > 0 else 0

```

Code Explanation: Evaluation Setup

- `model = search.best_estimator_`: Extracts the fully trained, refitted pipeline (including the preprocessor, SMOTE step, and tuned classifier).
- **Crucial Step:** We call `model.predict(X_test)`. Because `X_test` was *never* seen by the `fit` method, the SMOTE step in the pipeline is **skipped** during prediction. It only transforms the data and predicts.
- `fnr = fn / (fn + tp)`: We explicitly calculate the False Negative Rate. In a clinical setting, if FNR is 10%, it means 10% of sick patients are being sent home incorrectly. We want this as close to 0% as possible.

Hold-Out Test Set Results: Metrics Table

Model	Acc	Prec	Rec	F1	ROC-AUC	FNR
Random Forest	**0.9016**	0.8667	0.9286	**0.8966**	**0.9740**	**7.14%**
Extra Trees	0.8852	0.8182	**0.9643**	0.8852	0.9632	**3.57%**
XGBoost	0.8852	0.8889	0.8571	0.8727	0.9426	14.29%
Gradient Boosting	0.8525	**0.9130**	0.7500	0.8235	0.9578	25.00%
AdaBoost	0.8525	0.8800	0.7857	0.8302	0.9405	21.43%
Decision Tree	0.7869	0.7586	0.7857	0.7719	0.8523	21.43%

Observation

SMOTE significantly boosted Recall across the board compared to Week 5 baselines, but at the cost of some Precision.

Interpreting the Results: SMOTE vs Baseline

- **Recall (Sensitivity) Surge:** Extra Trees jumped to a massive **96.43%** Recall (FNR of only 3.57%). It missed only 1 sick patient out of 28 in the test set. This is a direct result of SMOTE giving the model more boundary examples of the minority class.
- **Precision Trade-off:** Extra Trees' Precision dropped to 81.82%. Because the model is now highly sensitive to the minority class, it flags more healthy patients as sick (False Positives increased from 4 to 6).
- **Random Forest Stability:** Random Forest maintained an excellent balance. Its Recall is still very high (92.86%), but its Precision (86.67%) and ROC-AUC (0.9740) are superior to Extra Trees, making it more reliable overall.

The Clinical Trade-off Revisited

Scenario A: "Miss No Cases"

Choose: Extra Trees

Why: Lowest FNR (3.57%).

Consequence: 6 healthy patients undergo unnecessary, stressful, and costly angiography. Healthcare budget strained.

Scenario B: "Optimal Reliability"

Choose: Random Forest

Why: Best F1-Score (0.8966) and ROC-AUC (0.9740).

Consequence: 2 healthy patients missed (FNR 7.14%), but the model is highly trustworthy and cost-effective for county facilities.

Recommended Final Model

Final Recommendation: Random Forest (with SMOTE)

For this study, the **SMOTE-augmented Random Forest** is selected as the overall final model.

- **Justification:** It achieved the highest hold-out ROC-AUC (0.9740) and PR-AUC, indicating superior discriminative power without overfitting to synthetic noise.
- Its Recall of 92.86% is exceptionally high for a clinical setting, successfully identifying 26 out of 28 actual disease cases.
- It avoids the severe precision drop seen in Extra Trees, ensuring the model remains trustworthy and cost-effective for resource-constrained county health facilities.

Part 7: Deliverables & Next Steps

"Consolidating today's learning."

Deliverable: Evening Lab Submission

Evening Lab Submission Requirement

Participants must submit a concise Jupyter Notebook or PDF report containing:

1. **Pipeline Summary:** Code for the `ImbPipeline` demonstrating the correct order: Preprocessor → SMOTE → Classifier.
2. **Leakage Prevention:** A brief written explanation of why `class_weight` was removed and how `ImbPipeline` prevents data leakage during CV.
3. **Hold-Out Evaluation:** Confusion Matrix, Classification Report, and ROC/PR curves for the **Random Forest** SMOTE model.
4. **Policy Memo:** A 1-paragraph justification for selecting Random Forest over Extra Trees, explicitly referencing the Precision/Recall trade-off and clinical costs (FN vs FP).

Common Pitfalls to Avoid

- **Pitfall 1: SMOTE on the whole dataset.**
  Fix: Always split first, then apply SMOTE only to `X_train`.
- **Pitfall 2: Using standard `sklearn.pipeline.Pipeline` with SMOTE.**
  Fix: Always use `imblearn.pipeline.Pipeline`.
- **Pitfall 3: Forgetting to remove `class_weight`.**
  Fix: Audit your hyperparameter grids to ensure no double-balancing occurs.
- **Pitfall 4: Evaluating SMOTE performance using only Accuracy.**
  Fix: Always prioritize Recall, F1, and PR-AUC for imbalanced clinical data.

Beyond SMOTE: Advanced Techniques

For Future Exploration

While SMOTE is powerful, it has limitations (e.g., it can generate noisy samples in overlapping class regions).

- **SMOTENC:** Specifically designed for datasets with mixed categorical and numerical features (avoids encoding artifacts).
- **Borderline-SMOTE:** Only generates synthetic samples near the decision boundary, reducing noise.
- **ADASYN (Adaptive Synthetic):** Generates more synthetic data for minority samples that are harder to learn (closer to majority class).
- **Tomek Links / ENN:** Undersampling techniques often combined with SMOTE (SMOTE + Tomek) to clean up the decision boundary.

Summary of the Data Balancing Workflow

1. **Analyze:** Check class distribution and decide if balancing is necessary (based on business/clinical cost, not just skew).
2. **Split:** Perform a stratified train/test split.
3. **Pipeline:** Build an `ImbPipeline` with Preprocessing $\rightarrow$ SMOTE $\rightarrow$ Model.
4. **Clean Grids:** Remove algorithmic balancing parameters (`class_weight`) from hyperparameter grids.
5. **Tune:** Run `RandomizedSearchCV` (SMOTE applies safely per fold).
6. **Evaluate:** Assess the best model on the pure, untouched test set using Recall, Precision, and PR-AUC.

The Bigger Picture: ML in Healthcare

Responsible AI

Machine learning in healthcare is not just about maximizing a metric; it is about **managing risk**.

- A model with 99% accuracy that misses 50% of heart attacks is a **failed** model.
- Techniques like SMOTE empower us to force the algorithm to pay attention to the rare, critical cases.
- However, we must remain vigilant against over-correction (destroying Precision), which erodes clinician trust and wastes hospital resources.
- **Balance is the ultimate goal.**

Questions & Discussion

Q&A

- How would you handle a dataset that is 99% imbalanced?
- What happens if SMOTE is applied to the test set by mistake?
- How do we explain "synthetic" patients to medical stakeholders?

References & Further Reading

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: synthetic minority over-sampling technique*. Journal of artificial intelligence research, 16, 321-357.
- He, H., & Garcia, E. A. (2009). *Learning from imbalanced data*. IEEE Transactions on knowledge and data engineering, 21(9), 1263-1284.
- `imbalanced-learn` Documentation: `https://imbalanced-learn.org/stable/`
- Scikit-learn User Guide: *Common pitfalls and recommended practices* (Section on Data Leakage).

Key Takeaways & Thank You!

- **Imbalance is a business problem, not just a data problem:** Choose metrics (Recall/PR-AUC) that reflect real-world costs.
- **SMOTE is powerful but dangerous:** It must be isolated within an `imblearn` Pipeline to prevent catastrophic data leakage.
- **Avoid double-balancing:** Remove `class_weight` when using data-level balancing techniques.
- **The Test Set is Sacred:** Never apply SMOTE, scaling, or imputation to the hold-out test set.

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College | Nairobi, Kenya
Email: lumumbavictor172@gmail.com